

Degree Project in Machine Learning

Second cycle, 30 credits

Latent grid distributions for better robustness in Beat Tracking

Encoding ambiguity with metric distributions over a learned tatum grid

ROBERT KIHLBORG

Latent grid distributions for better robustness in Beat Tracking

Encoding ambiguity with metric distributions over a learned tatum grid

ROBERT KIHLBORG

Master's Programme, Machine Learning, 120 credits
Date: September 18, 2026

Supervisors: Andre Holzapfel, Jan Schlüter
Examiner: Bob Sturm
School of Electrical Engineering and Computer Science
Host organization: Johannes Kepler University Linz
Swedish title: Underliggande fördelningar över rytmiska tolkningar för robustare taktanalys
Swedish subtitle: Beskrivningar av tvetydlighet genom fördelningar över rytmik ovanpå en inlärd puls

© 2026 Robert Kihlborg

Abstract

Beat Tracking is the task of predicting the positions of beats and/or downbeats in a musical recording. The current state of the art uses Deep Learning models on spectrogram representations of music to estimate beat and downbeat positions. Usually, the networks produce a beat activation curve that represents the probability of a beat, and then post-process that curve to get the final estimates. To ensure musical consistency, this is often done with Dynamic Bayesian Network (DBN) postprocessing, which only works for certain types of music. Finding DBN-free ways to enforce musical consistency is desirable as it could expand the usefulness of Beat Tracking algorithms. This thesis explores novel ways of using theory-informed modules in tandem with Deep Learning to facilitate learning of metrical consistency. The proposed model predicts a tatum grid: a regular grid of feasible beat candidates, and a subgrid: a regular subset of the tatum grid that constitutes the final estimation. The subgrid treats meter like a classification problem, allowing for multiple hypotheses for the time signature and tempo, which can better represent ambiguity than a prediction based on local beat probabilities. The trained model improves continuity metrics compared to state-of-the-art DBN-free algorithms, though it still performs worse than DBN-methods on the test dataset. Despite good scores, qualitative analysis shows the model sometimes failing on easy data, suggesting that the algorithm has a lot of room for improvement.

Keywords

Beat Tracking, Music Informatics, Machine Learning

Sammanfattning

Taktanalys går ut på att automatiskt hitta takten i inspelad musik. De bästa algoritmerna idag transformerar musiken till spektrogram och använder dessa som indata till djupinlärningsmodeller som försöker lära sig att placera ut taktslag på rätt ställen. Vanligtvis görs detta genom att först producera en signal som vid varje tidpunkt representerar den lokala sannolikheten för ett taktslag, som sedan kan omvandlas till en fullständig gissning. För att omvandlingen mellan taktsignalen och slutgiltiga gissningen ska vara musikaliskt konsekvent används ofta Dynamiska Bayesianska Nätverk (DBN) som kan garantera konsekventa uppskattningar givet begränsningar inom exempelvis tillåtna tempon eller taktarter. Genom att göra sådana antaganden så kan modellens snittprestanda höjas, men det innebär också att hela genrer av musik exkluderas. Därför är det värdefullt att hitta DBN-fria algoritmer som kan förutspå konsekventa takter. Det här examensarbetet utforskar nya sätt att använda moduler inspirerade av musikteori tillsammans med djupinlärning för att underlätta inlärning av konsekventa taktgissningar. Den framtagna modellen börjar med att förutspå ett tatumnät: en regelbunden serie av taktkandidater, och ett delnät: en regelbunden delmängd av tatumnätet som utgör den slutgiltiga gisssningen. Delnätet tolkar taktart som ett klassifikationsproblem vilket möjliggör för flera hypoteser för tempo och taktart, vilket gör den bättre på att uppskatta tvetydligheter än DBN-modeller som bara producerar en tolkning. Modellen presterar bättre än andra DBN-fria modeller på tester som mäter konsekvent korrekthet, men presterar sämre än de bästa DBN-modellerna i snitt. Trots bra mätvärden visar den kvalitativa analysen att modellen har oväntade svårigheter på relativt enkelt indata, vilket antyder att algoritmen har enkla förbättringsmöjligheter.

Nyckelord

Taktanalys, Musikinformatik, Maskininlärning

Acknowledgments

I would like to thank my supervisor at JKU, Jan Schlüter, and my KTH supervisor André Holzapfel for their help, collaboration and friendship over the course of this project. I would also like to thank my examiner Bob Sturm, and finally, all of my colleagues at the Computational Perception division at JKU.

Stockholm, September 2026
Robert Kihlborg

Contents

1 Introduction **1**
1.1 Background . . . 1
1.2 Problem . . . 4
1.3 Purpose and goals . . . 5
1.4 Research Methodology . . . 5
1.5 Delimitations . . . 5
1.6 Disclaimer of Originality . . . 5

2 Background **7**
2.1 Beats and meter in music . . . 7
2.2 Musical features . . . 8
2.3 Deep Learning in Beat Tracking . . . 8
2.3.1 Regression and Classification . . . 9
2.3.2 Beat activation . . . 10
2.3.3 Regularization . . . 10
2.4 Related works . . . 11
2.4.1 Comparison models . . . 12

3 Method **13**
3.1 Design process . . . 13
3.2 Model . . . 13
3.2.1 Grid . . . 14
3.2.2 Subgrid . . . 17
3.2.3 Inference . . . 19
3.3 Data . . . 19
3.3.1 Data quality issues . . . 19
3.4 Training setup . . . 20
3.5 Evaluation methodology . . . 20

4 Results and Analysis **25**
4.1 Major results 25
4.1.1 Grid evaluation 25
4.1.2 Beat and Downbeat Tracking evaluation 27
4.1.3 Qualitative analysis 28
4.2 Reliability Analysis 29

5 Discussion **33**

6 Conclusions and Future work **35**
6.1 Conclusions 35
6.2 Future work 36
6.3 Reflections 36

References **39**

List of acronyms and abbreviations

BCE	Binary Cross-Entropy
CNN	Convolutional Neural Network
DAW	Digital Audio Workstation
DBN	Dynamic Bayesian Network
DL	Deep Learning
dPLP	differentiable Predominant Local Pulse
EMD	Earth Mover's Distance
HMM	Hidden Markov Model
LSTM	Long-Short Term Memory
MIR	Music Information Retrieval
MoE	Mixture-of-Experts
MSE	Mean Squared Error
NN	Neural Network
PLP	Predominant Local Pulse
STFT	Short Time Fourier Transform
TCN	Temporal Convolutional Network

Chapter 1

Introduction

Beat Tracking is one of the core areas of Music Informatics. Automatic annotation of beats and downbeats has use cases in academia and industry. For example, it can be used in musicology to study how rhythmic expression differs between cultures and how it has changed over time. It can also be used in Digital Audio Workstations (DAWs) to automatically extract tempo and time signature information from a recording, which is very helpful for music producers. Finally, almost all algorithms for music analysis will implicitly depend on rhythm, so improvements in Beat Tracking can be used to improve other areas of the field.

1.1 Background

Modern beat tracking generally follows the strategy outlined in Figure 1.1:

1. The musical recording, in the form of a digital signal, is transformed to a spectrogram using the STFT.

2. The spectrogram is split into pieces of a fixed length (for example 30 s), ensuring a consistent dimensionality.

3. Every piece is fed into a Neural Network that produces an activation function that can be compared to the true location of beats and downbeats. The difference is minimized during training.

4. The pieces are postprocessed and then combined, or combined and then postprocessed. The result is a pair of prediction masks.

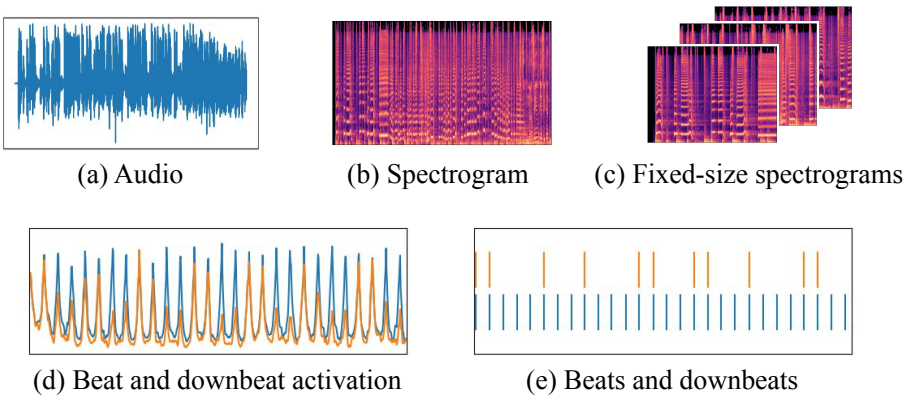
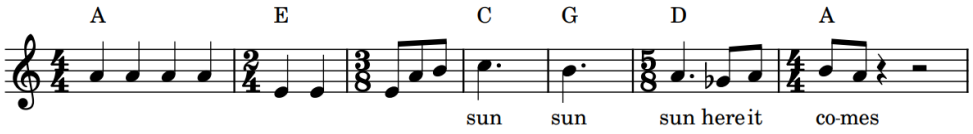


Figure 1.1: Summary of the general strategy used by most beat trackers. Audio (a) is transformed with the Short Time Fourier Transform (STFT) to a spectrogram (b). This spectrogram is split into fixed size segments (c). These segments are fed into the model to produce the beat and downbeat activation (d). This is postprocessed into a final prediction (e).

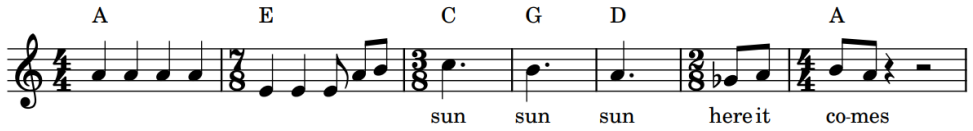
Postprocessing of the framewise beat activations can be done in several ways. The simplest is thresholding, which means that a beat is predicted at each local maximum of the framewise activations that has a value greater than a fixed threshold. Dynamic Bayesian Network (DBN)-postprocessing, on the other hand, interprets the framewise activations as observations from a Hidden Markov Model (HMM). By specifying the set of allowed states, transition probabilities and observation probabilities, an internally consistent estimate can be extracted. This comes with the major downside of having to define the parameters of the HMM. Using HMM parameters that perform best on a specific dataset can lead to overfitting, for example by only allowing certain time signatures and disallowing time signature changes.

Predictions are commonly described by a prediction mask, which is a boolean vector where every entry corresponds to a short time window, and a beat is represented as a 1 in a specific time slot in the vector. Predictions can also be expressed as lists of numbers representing time stamps of beats, but this requires dynamic sizing which is less useful for computational implementations.

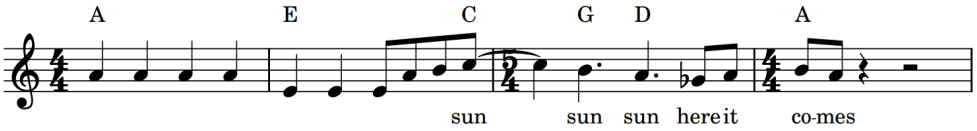
Beat Tracking can be regarded as inherently stochastic because there can be multiple valid ways to interpret a piece, for example in the bridge of Here Comes the Sun by the Beatles (Figure 1.2). Therefore, the truth for each



(a) The interpretation in the annotated version from the Beatles [1] dataset.



(b) Another feasible interpretation that has the same beats but different downbeats.



(c) Another possible interpretation that does not use a faster beat.

Figure 1.2: Three different metric interpretations of the bridge in Here Comes the Sun by the Beatles. There is no single truth, though some interpretations can be more reasonable than others. Here, (a) and (b) are both valid while (c) contains unexpected drum accents and would probably not be considered correct by most listeners.

musical piece should really be a distribution over interpretations, though it would be very impractical to annotate it in practice. With enough training data, a model can be expected to learn to express this stochasticity in the framewise activations. This exposes a fundamental weakness, as a distribution over interpretations cannot be encoded as the framewise probabilities. An encoding function needs to be invertible, but different distributions over interpretations will map to the same framewise activations, as shown in Figure 1.3.

The mathematically ideal way to express a prediction would be a distribution over every possible set of beat and downbeats. This is completely impractical as it would be far too high dimensional for any computer to handle. The framewise activations represent the opposite extreme, as it assumes that the distribution can be factored as a set of local independent probabilities.

This thesis addresses these shortcomings by designing a novel architecture that allows the model to keep a richer encoding than is possible with only framewise activations, while using musical assumptions to limit dimensionality. It starts with the assumption that all feasible beats will exist on a *tatum grid*: a regular pulse train. Then, the tempo, time signature, and phase can be expressed locally as a categorical distribution over subsets of the

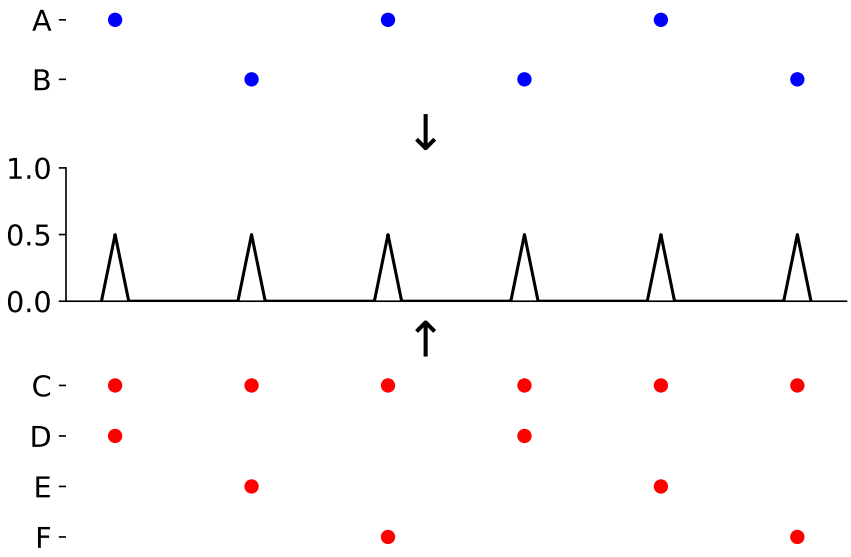


Figure 1.3: Illustration of how independent framewise probabilities can never encode ambiguity. A sample that is ambiguous between A and B (top) will produce the same local probabilities (middle) as a sample that is ambiguous between C, D, E and F (bottom).

tatum grid, called the *subgrid*. This greatly reduces dimensionality, while also allowing lossless expression of ambiguity.

The open-source Beat This model by Foscarin *et al.* [2] will be used as a starting point, with the innovations being inserted on top of the existing Neural Network (NN) architecture.

1.2 Problem

We will try to determine whether our proposed strategy can be useful for Beat Tracking by answering the following questions:

1. Can a neural network learn to predict a tatum grid, for which there is no ground truth?

2. Can more consistent musical beat predictions be achieved by expressing a prediction as a distribution over meters on the learned tatum grid?

1.3 Purpose and goals

The purpose of this work is to explore new avenues and further research on Beat Tracking. Hopefully, the results and lessons learned from this project will inspire and guide future research.

The primary goal is to design and document a model that follows the proposed strategy, and share discovered advantages and difficulties. The secondary goal is to show that the model can perform better under certain circumstances, measured with standard metrics on public datasets.

1.4 Research Methodology

The design process for the model will be based on theory, qualitative analysis and quantitative analysis. First, existing solutions will be studied. Then, small tests will be conducted on simulated data to study the viability of the design, before moving on to experimenting on training data consisting of beat-annotated music recordings. Qualitative analysis will be performed in the form of looking at and listening to predictions on training and validation data, to understand where the design fails. Quantitative analysis will be conducted through measuring metrics on the validation set during training.

As with all Machine Learning research, a key part of the methodology is to not let information about the test dataset influence the design of the model, as that introduces a bias. Therefore, all design decisions and parameter choices will be based on validation performance, with the test set being invisible to the model until the final test.

1.5 Delimitations

This thesis focuses on exploring the new grid-subgrid strategy and will not consider all confounding design choices. Standard datasets and metrics will be used, and the majority of the NN architecture will be taken directly from the Beat This [2] model. Ablation studies with non-learned methods of predicting a tatum grid for the model will not be conducted.

1.6 Disclaimer of Originality

This thesis is based on a project that also resulted in an ISMIR paper [3]. That paper was written in collaboration with my supervisors, while this thesis

text is solely the work of me, with the exception of some figures taken from the paper. Though the underlying findings are largely the same, this paper is meant to serve as a deeper and more thorough explanation of the model than was possible within the six-page limit of ISMIR. Finally, I use **we** instead of **I** in this thesis because it reads better, not because I am speaking for anyone else.

Chapter 2

Background

This section will explain the musical concept of beats and downbeats, and how it can be translated to a machine learning problem. The existing work in the area will be presented and explained.

2.1 Beats and meter in music

In music, the beat refers to the perceived regular pulse that listeners intuitively synchronize with, for example by tapping their foot or clapping along with a song. Beats provide a temporal framework for organizing musical events and are central to the perception of rhythm. While the beat is often associated with physical acoustic events such as drum hits, it is fundamentally a perceptual construct: listeners may infer beats even when they are not explicitly articulated in the audio signal. Consequently, beat perception depends both on acoustic cues and on cognitive expectations formed through rhythmic regularity and musical context.

Multiple beats can be grouped together to form a measure, which represents the closest hierarchical level above beats. The first beat in a measure is usually more accented, and is called the downbeat. Beats can also be subdivided into faster metrical structures, where the tatum describes the fastest subdivision that exists in the music. The tatum can be musically ambiguous, and comes from Music Information Retrieval (MIR) rather than musicology, with the term being coined by [4]. It is a practical assumption that all musical events in a piece will be quantized in terms of a fast pulse, though it is not always true, and not always obvious how fast the tatum needs to be. For this project, tatum will be used as a proxy for a pulse that encompasses all feasible interpretations of beats. This is more in line with the musicological

descriptions provided by [5], and it is more useful from a beat tracking standpoint.

2.2 Musical features

Beat Tracking from audio signals has been studied since at least 1994 with an early notable algorithm by [6] by Goto *et al*. This algorithm tried to solve the problem in real time. Due to the limited compute and data at the time, state-of-the-art performance was achieved by making strong assumptions about the music and hand crafting ways to detect certain musical features. In the case of Goto *et al.*, this was achieved by trying to extract the bass and snare drums, and assuming a time signature of 4/4 with a bass-snare-bass-snare pattern.

Following papers have used other kinds of hand-crafted features, often calculated from a spectrogram and focused on extracting onsets as instants with a sudden changes in the time or frequency domain signal. Such change-based features are good for music with clear and common rhythmic patterns, but inevitably fail for pieces with strange, unexpected or missing rhythms. It also cannot take into consideration non-rhythmic features such as harmony or lyrical context, which can be very important to a human listener.

2.3 Deep Learning in Beat Tracking

The general strategy used for Beat Tracking with Deep Learning was discussed in section 1.1. The use of Deep Learning for Beat Tracking has become dominant due to improvements to three factors: the increased availability of compute due to CPU and GPU improvements, the growing amount of annotated training data from the global research community, and improved architecture design. The feedback loop of improvement sparking other improvement has been shared across the entire field of Machine Learning, and progress has largely been shared between similar fields. For example Long-Short Term Memory networks (LSTMs), Convolutional Neural Networks (CNNs) and Transformers have been involved in recent state-of-the-art Beat Tracking models, though they were originally designed for other applications.

Deep Learning methods can generalize far better than hand crafted features, especially when non rhythmic-context has to be taken into consideration. However, it is susceptible to common Deep Learning (DL) problems like overfitting. This is especially important as music comes in different genres, and beyond overfitting to the training data, the model can

overfit to the genre composition in the training data, even if validation is handled well.

2.3.1 Regression and Classification

An implicit subtask in Beat Tracking is the ability to estimate bpm and beat phase from likely beat positions, which can be thought of as fitting a regular grid to a set of points. Though a one-dimensional grid is fully described by two scalars (frequency and offset, for example), learning to predict these is surprisingly difficult for a Neural Network. The reason for this is that predicting a grid behaves as both a regression and a classification problem simultaneously. Regression in the sense that close to the optimum phase and frequency, a reasonable loss will be convex. Classification in the sense that there is usually an integer number of feasible interpretations where the frequency differs by an integer factor.

As an example, if a model estimates a tempo of 120 bpm and the ground truth turns out to be 60 bpm, we would not want the model to push the guess closer to the target and estimate 110 bpm next time as that would be more incorrect. Instead, we need it to be able to shift its confidence from 120 to 60 bpm, like a classification network. However, if the model estimates 120 bpm and the truth is 122 bpm, then we just want to shift the 120 bpm guess towards 122 bpm, like in a regression network. This can be achieved by using some architecture with multiple hypotheses, like a Mixture-of-Experts (MoE) model.

Another way to try to solve this problem is to stay fully in the classification realm by discretizing the prediction space, for example by having a separate bin for every possible bpm 60 to 200. However, when the model needs to jointly predict bpm, beat phase, downbeats per beat and downbeat phase, the number of bins becomes far too high with this approach, and training becomes infeasible. Another disadvantage of solving a regression problem with classification is that the ability to generalize is lost. A regression network that has never seen a song between 74 and 76 bpm can theoretically extrapolate from other data and correctly identify a 75 bpm song, while a classification network that has not seen a specific class will be trained to never guess that class.

Most models avoid this problem completely by never directly trying to predict the beat grid directly. Instead, the hidden layers of the network are expected to learn to encode rough metric structure, as it helps with predicting the actual beat positions.

2.3.2 Beat activation

The idea of a beat activation function was mentioned in 1.1, but merits a deeper explanation. Common to most DL approaches is having a main part of the model M that takes the input x (usually a spectrogram) and computes $\alpha = M(x)$ as a time-vector that is high in areas with high likelihood of beats and vice versa. Here, α is the beat activation, and every entry in α corresponds to some short window on the order of 10 ms. The task loss then uses α and the target beats y to train the model. For inference, estimated beats y' are computed from α using some decoding strategy. The crucial detail is that the loss does not directly use y', as computing $y'(\alpha)$ generally is not differentiable, and therefore cannot be used with backpropagation. This means that the model is only directly trained to predict α, not y'.

This strategy is common in Machine Learning and is not a problem in itself. Any basic classifier model predicts distributions that are used for the loss, and then has to use a decoding strategy to sample from that distribution to get the final output. However, for more complicated tasks this comes with the challenge of designing a loss and a decoding strategy that work well together, and this design is one area where models differ. Depending on that choice, the beat activation will represent different things and look different.

A basic way to train a model to predict a beat activation is to push every entry in α to predict the probability of a beat during that frame. A naive decoder could then create a prediction y' by independently sampling from every frame. This would be a terrible idea as beats are very heavily interdependent. A decoder for this α would need to be complicated and take this interdependence into consideration. A smarter loss can offset some of this complexity and allow for a simpler decoder, as in the case of [2], but it cannot fully compensate for the inherent weakness in the beat activation architecture.

2.3.3 Regularization

The term regularization originally referred to the idea of penalizing large parameter values to decrease overfitting, but has grown to include other measures to increase generalization performance by pushing the system in another direction than the task loss. We will use regularization loss to refer to parts of the loss that only depends on the model prediction x and not the target value y, as seen in equation 2.1.

$$L(x, y) = L_{task}(x, y) + L_{reg}(x) \tag{2.1}$$

2.4 Related works

Models using a beat activation task head and deterministic postprocessing have been studied at great length, with the postprocessing-DBN proposed by Böck *et al.,* in [7] and tuned in [8] being a de-facto standard. The underlying network architecture has been the subject of several works, with [8] using LSTM. Böck *et al.* [9] achieves better performance by switching to Temporal Convolutional Networks (TCNs), [10] by Hung *et al.,* proposes the Transformer based SpecTNT architecture, and [11] by Zhao *et al.,* also uses transformers in addition to demixing the input spectrogram by instrument.

Alternative task heads, i.e. the choice of loss function and postprocessing, have also been studied. The TCN-based [12] by Chen *et al.,* uses Focal loss and Dice loss along with phase estimation to minimize the required postprocessing, predicting beats at local maxima of the resulting beat activation function. [2] by Foscarin *et al.,* is based on Transformers, and proposes the Shift Tolerant Binary Cross Entropy loss that similarly allows it to replace DBN with simple local-maxima postprocessing.

Some research has been done on computing and using tatum grids, for example in the pre-Deep Learning Beat Tracker [13] by Seppänen. [14] uses the autocorrelation function to determine the frequency of the tatum, and finds the frequencies of higher metrical structures by looking at integer divisors of the tatum frequency, also without Machine Learning. In [15] by Durand *et al.*, the tatum is used as a first step in a Deep Learning Beat Tracker. In the latter, the tatum is found using Predominant Local Pulse (PLP) [16], which is a deterministic algorithm to find find a pulse from an onset function. differentiable Predominant Local Pulse (dPLP) [17] proposed by Chiu *et al.,* is designed to allow backpropagation which allows it to be used in Deep Learning, but the algorithm itself is still deterministic.

The closest approach to the one we propose is [18] by Gagneré *et al.* It finds the tatum with PLP, and then uses contrastive learning, a kind of MoE strategy, to express multiple hypotheses of beats over the tatum. One issue that arises is that PLP has strong requirements on regularity and cannot handle changing tempos, so pieces with changing tempos are discarded from the training of [18]. The PLP is also only used during pretraining, and discarded in favor of a beat activation approach during the fine tuning and testing.

As far as we know, there has been no published research into learning a tatum grid with Deep Learning.

Model	About
Beat This [2]	Beat Activation, DBN-free
Beat This + DBN	Beat Activation, DBN from [7]
Gagneré *et al.* [18] ST-BCE	Pretrained with PLP and multiple hypotheses, DBN-free
Gagneré *et al.,* DBN	Pretrained with PLP and multiple hypotheses, DBN from [7]
Hung *et al.* [10]	Beat Activation, DBN from [7]

Table 2.1: Summary of comparison models.

2.4.1 Comparison models

For the tatum grid, we will compare our algorithm against PLP. An overview of the models that we will compare our final model to is given in Table 2.1.

Chapter 3

Method

3.1 Design process

The specifics of the model design were not predetermined as the project started, but rather discovered as a series of adaptations during the exploration stage. Environments with very simple synthetic data were built to test the feasibility of each of the components of the model, and this was instrumental to the success of the design process.

This was particularly important for the grid estimation, which had to deal with the challenges described in Section 2.3.1. The convexity of the loss could be tested by choosing a fixed target, and then trying to learn to predict that target using gradient descent on the loss, letting the model learn a single tensor. Though primitive, this caught a lot of problems and influenced the design of the grid module.

3.2 Model

An overview of the proposed model is shown in Figure 3.1. The underlying architecture is taken directly from Beat This [2] by Foscarin *et al*. The model takes a 30-second musical excerpt that is transformed to a spectrogram. The spectrogram is fed through a frontend with convolutional layers and then six transformer blocks. In the middle of the transformer blocks, we insert the Grid module, which is tasked with predicting the tatum grid, and then feeding the prediction back into the system. Finally, the Subgrid block predicts the distribution of meters over the tatum grid, with the total model output consisting of the tatum grid and the subgrid prediction for each grid point. The structure of the model output will be further explained in section 3.2.2.

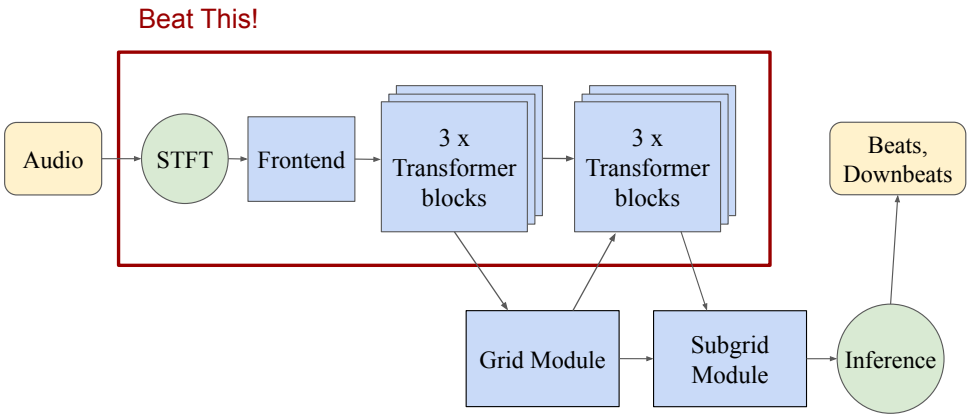


Figure 3.1: The general architecture of the model. The base comes from Beat This [2], and we have added the Grid and Subgrid modules and the corresponding new inference. The blue rectangles constitute the learned parts of the model.

For inference, the model output can be postprocessed with Thresholding or Viterbi to get the final beat and downbeat predictions.

3.2.1 Grid

The grid module receives a 256×1500 input from the transformer blocks, with 256 being the number of channels in the transformer and 1500 being the number of time steps for a 30 second excerpt. The task of the module is to produce a 1×1500 boolean vector of tatum grid points. Secondarily, it also produces a 2×1500 phase signal in the form of a sine and cosine wave that are $(0, 1)$ at grid points and continuously interpolate the phase, making one full revolution between tatums. This auxiliary signal is useful as it is a dense representation of the frequency and speed of the tatum, rather than the sparse representation carried by the boolean vector.

For the tatum grid to be useful for downstream use, we need it to fulfill three design criteria:

1. The grid should achieve close to 100% recall on the ground truth beats, as any beat missed by the tatum will be missed by the final prediction.

2. The grid should not represent a faster-than-necessary subdivision of the beat, as it increases the downstream dimensionality required to describe

the beat in terms of the tatum.

3. The grid should reflect a consistent metrical level unless the metric feel changes drastically. Jumps in metrical level would be difficult to handle downstream.

We choose to express the grid in terms of tempo and phase rather than framewise probabilities, as this makes it easier to ensure consistency. To allow for a dynamic tatum grid that can stretch and change speed, we split the excerpt into 1-second windows and predict the two grid parameters, tempo and phase, independently in each of them.

From the tempo and phase, we compute a sine wave inside every window, and use the transformation $f(x) = (x+1)/2$ to move the sinusoid to the range $[0, 1]$. The peaks representing the corresponding grid points. We define the *Soft Recall* loss as the mean of the sinusoid evaluated at every true beat. The soft recall is useful as it directly targets recall but ignores precision. This is important as we want to allow the tatum to be faster than the beat, which results in a low precision. However, it does have a slight preference for higher precision, as a slower tatum means wider sine wave peaks and therefore less susceptibility to slight timing errors. The soft recall directly solves design criteria (1) and (2).

Using soft recall as a training objective presents a major problem, however. The loss is not convex, it has several minima, some corresponding to reasonable estimates of tatum and others not. As discussed in section 2.3.1, this is because predicting a grid acts as both a classification and a regression problem. An illustration of this is shown in Figure 3.2.

The solution to the convexity problem is to utilize a MoE strategy where for every prediction window, the model guesses different grids in different frequency bands, alongside a confidence for each. As long as the frequency bands are narrow enough, this ensures convexity within the correct band. We chose 12 frequency bands each covering a third of an octave, with a total cover of 4 octaves between 30-480 pulses per minute. The total soft recall loss is the weighted sum of the individually estimated losses, using the softmax of the predicted confidences as weights.

The window-based prediction also introduces the issue of ensuring the grid is consistent. If we consider the case with a single estimate per window, we can define a distance between two grid estimates based on the frequencies and

the phase jump:

$$\begin{aligned} D(f_1, f_2, \Delta\theta) &= D_f(f_1, f_2) + D_\theta(\Delta\theta, f_1) \\ D_f(f_1, f_2) &= \alpha \left|\log(f_1) - \log(f_2)\right| \\ D_\theta(\Delta\theta, f_1) &= \frac{\beta}{f_1}\left(\sin(2\pi\Delta\theta)^2 + (1 - \cos(2\pi\Delta\theta))^2\right) \end{aligned} \tag{3.1}$$

α and β are hyperparameters with default values of $\alpha = 2/50$, $\beta = 1/(50 \log 1.05)$.

When we have multiple estimates per window we are dealing with discrete distributions over phase frequency-space, and we can define the window discrepancy as the Earth Mover's Distance (EMD) between the distributions, using the pairwise distances from Equation 3.1. We then use this in a regularization loss that penalizes mean window discrepancy. The EMD is calculated with the Sinkhorn algorithm [19] with a fixed number of iterations.

When computing the window discrepancy like this, the loss is easily dominated by very large distances between grid candidates at the opposite ends of the frequency range. To combat this, we limit the pairwise distance with a soft clamping function:

$$softclamp(x) = 2v(\sigma(x/v) - 0.5) \tag{3.2}$$

which is a differentiable way to clamp positive real numbers to the range $[0, v)$. σ refers to the sigmoid function and $v = 2$ is a hyperparameter.

In summary, for each adjacent pair of windows, we compute a matrix of pairwise distances between the tatum grid candidates using Equation 3.1. We then apply the softclamp function from Equation 3.2 to every value in the matrix. The Sinkhorn algorithm is used to compute the EMD between the candidate distributions calculated from their respective confidences, with the clamped pairwise distance matrix. The resulting value is the window discrepancy between the pair of windows. The mean window discrepancy is penalized in the regularization loss.

To get the final output, we concatenate the sinusoids for the most probable candidate grids in each window and then apply a gaussian kernel to reduce the risk of double peaks at window edges. Then we pick all local maxima as the estimated grid points.

In summary: the grid is described by 30 one-second tatum analysis windows calculated from the corresponding 256×50 transformer output through three linear layers of 50 neurons with GELU activation. Each window

is described by a 12×3 tensor, describing frequency, phase and confidence for each of the 12 candidate grids (Figure 3.3). We use the weighted mean of the soft recall as the training objective, and additionally introduce a regularization loss to make sure adjacent windows are consistent. Finally, we concatenate the sinusoids corresponding to the most probable grid candidates from every window, apply smoothing and then pick local maxima as the estimated grid points. This results in a 1×1500 boolean mask of grid points, together with a 2×1500 smoothed grid wave containing sine and cosine.

The boolean mask is passed on for direct usage in the downstream, and the mask along with the waves are projected to 256 dimensions and added to the existing transformer output to be passed on to the subgrid.

This strategy was susceptible to expert collapse. This happens when the model is most confident in one specific candidate. That leads to the loss being weighed higher for the regularization and training of that specific candidate, which leads it to improve faster which in turn leads to it being weighed even higher. The model then has no incentive to confidence in other candidates as they are still untrained, and they cannot be trained as their confidence is close to zero. To break out of this catch 22, we add some uniformity to the probabilities of the grid candidates in the backwards pass. This makes sure that all candidates are updated in the backwards pass, which helps the model utilize all candidates.

3.2.2 Subgrid

The subgrid receives a 256×1500 tensor that encodes information about the grid as well as the underlying music. The task of the subgrid is to estimate the meter of the music relative to the tatum grid. To do this, it needs to estimate the integer phase and period of beats in terms of grid points, as well as integer phase and period of downbeats in terms of beats. This means that a single subgrid state at a grid point is described by a tuple of four integers: $(T_B, \varphi_B, T_D, \varphi_D)$ where $0 \leq \varphi_B < T_B$ and $0 \leq \varphi_D < T_D$. Here, T_B refers to beat period, φ_B refers to beat phase and analogously for the downbeats. If we assume upper bounds of $T_B \leq 4$ and $T_D \leq 7$, we limit the state space to 280 different metric possibilities. A smaller example can be seen in Figure 3.4.

An example of the subgrid handling ambiguity can be seen in Figure 3.5, where the model estimates probabilities for two different feasible tempos, one being twice the tempo of the other.

The subgrid block projects the 256×1500 transformer output to a 280×1500 framewise state-distribution. The marginal probabilities of a beat or

downbeat can be calculated by summing the probabilities of all states that have a 0 as the corresponding phase. We snap every ground truth beat and downbeat to the closest estimated tatum grid point, and then compare estimated beat and downbeat probabilities with the ground truth at every grid point, essentially discarding the transformer output at non-tatum points. The training objective is to minimize the difference between these, which we measure with Mean Squared Error (MSE). This would normally be done with Binary Cross-Entropy (BCE) which is designed to work with distributions, but that led to errors during training. This is discussed further in Chapter 5.

We have to snap the ground truth to estimated grid points to allow us to compare, but it can result in a strange loss when the estimated tatum grid is inaccurate. In general, training the subgrid is very unreliable when the tatum grid does not represent a consistent subdivision of the true beats. This is unavoidable with our architecture, and we found that assuming the tatum grid was good when designing the subgrid training worked well. When training from scratch, this also means that the subgrid essentially cannot learn anything until the tatum grid estimates are reasonably good, but that did not seem to cause problems.

As the model predicts states independently, we have to introduce a regularization loss to ensure consistency between the predictions at adjacent grid points. There is a bijection from every state to the implied following state:

$$(T_B, \varphi_B, T_D, \varphi_D) \rightarrow (T_B, (\varphi_B + 1) \,\%\, T_B, T_D, (\varphi_D + 1) \,\%\, T_D) \tag{3.3}$$

which can be used to predict the next state-distribution from the current. By taking the KL-divergence between the implied and actual distributions for every pair of neighboring tatum points, we get a good metric of internal consistency. This metric does not allow for changing time signatures, however, so we make a special case for downbeats (all states with $\varphi_D = \varphi_B = 0$) which are combined into a single bin in the divergence calculation. This allows the subgrid to freely change time signature at downbeats.

In summary, the subgrid projects the 256×1500 transformer output to a 280×1500 framewise distribution over subgrid states. This is masked by the estimated grid-mask, discarding all non-tatum frames. The ground truth is snapped to the estimated tatum, and used to calculate the MSE loss against the estimated beat and downbeat probabilities. A regularization loss is calculated by compared the estimated distribution between neighboring grid points, penalizing unexpected jumps in meter at non-downbeat states. The distribution over subgrid states can then be used by the inference stage.

3.2.3 Inference

Our model can be configured with two different ways to calculate final beat outputs from the subgrid state distribution. The primary way that the model is designed for is to employ the Viterbi algorithm to find the most likely series of states, given the subgrid estimate of the distribution at every grid point. We know all expected state changes from Eq. 3.3, which we can combine with some uniformity in the transition probability matrix to allow for jumps.

Another way is to take the framewise beat and downbeat probabilities and simply apply a threshold value. This is similar to what would be done in a framewise-activation based model, and it essentially treats the subgrid into a regularizer that mimics the effects of a DBN.

For our experiments, we used a random-jump probability of 0.3 for Viterbi inference and a threshold value of 0.5 for threshold inference.

When running on pieces longer than 30 seconds, the model will predict independently on each 30 second window and then concatenate the beats and downbeats. This is not ideal when using Viterbi inference as it potentially allows for unwanted metric jumps at window borders. The tatum grid and subgrid states should be concatenated before running Viterbi, but we did not have time to implement this.

3.3 Data

We used the exact same training datasets as Foscarin *et al.,* in [2]: ASAP [20], Ballroom [21, 22], Beatles [1], Candombe [23], Filosax [24], Groove MIDI [25], GuitarSet [26], Hainsworth [27], Harmonix [28], HJDB [29], JAAH [30], Simac [21], RWC [31, 32] (classical, pop, royalty-free, and jazz), SMC [33], TapCorrect [34]. Using the same datasets allows for a very direct comparison, which is useful. The datasets also span a wide range of musical styles and genres, including classical, jazz, ballroom and pop. In total, the datasets contain 4556 pieces. Simac and SMC do not contain information about downbeats, and were not used when calculating downbeat metrics.

For evaluation, we use GTZAN [35] as a test set that is not visible to the model during training.

3.3.1 Data quality issues

As noted by [2], there are some data quality issues with these datasets. In addition simple timing errors where the annotated beats are very clearly at

an offset, there are also inconsistencies in the annotation style, with certain 6/8 pieces being counted on 1 and 4, with beats 2,3,5 and 6 missing from the annotations. We ignored these issues, as it let us better compare the architecture to Beat This.

3.4 Training setup

We based our training setup on the open-source implementation provided by [2]. This includes the data-augmentation like varying tempo and pitch, sampling multiple times from pieces longer than 30 seconds, and using the same validation data splits. The training was done in 100 epochs in batches of 8. We used the AdamW optimizer with 1000 steps of Cosine Warmup and a learning rate of 0.0008.

3.5 Evaluation methodology

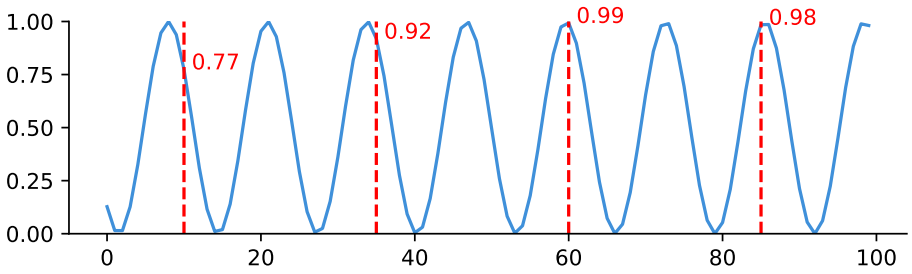
For evaluation, we ran the model on annotated music recordings, and compared the model output against the true beats. We measured both the quality of the estimated tatum grid and of the actual predicted beats and downbeats.

The tatum grid estimation presents a novel task without established standard metrics. We choose to use recall on beats as the primary metric for the tatum grid, and use the standard 70 ms margin of error. As a secondary metric, we use AMLt as it helps us detect if the tatum grid is too fast. Often, the desired tatum grid is twice the true tempo in which case the AMLt also should give a good score. AMLt should not be used as a point of comparison as other tatum-methods may favor faster grids, but it can be insightful.

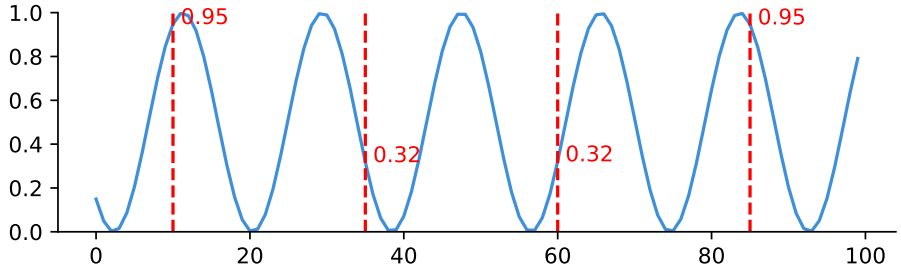
For the beat and downbeat estimates, we used standard Beat Tracking metrics provided by the *mir_eval* package [36], primarily F1-score, AMLt and CMLt. We use CMLt as the main metric, as it best captures when the prediction is not only correct but also consistent. AMLt is similar, but it is designed in a way that favors music where the meter can be subdivided into groups of two, and music with a constant time signature and tempo.

First, we trained one set of eight models using cross-validation on the training data, and let every model predict the excerpts in the corresponding validation split. This way, we got a blind prediction on every piece in the training set. From this, we calculated per-dataset cross validation metrics for the tatum grid, beats and downbeats.

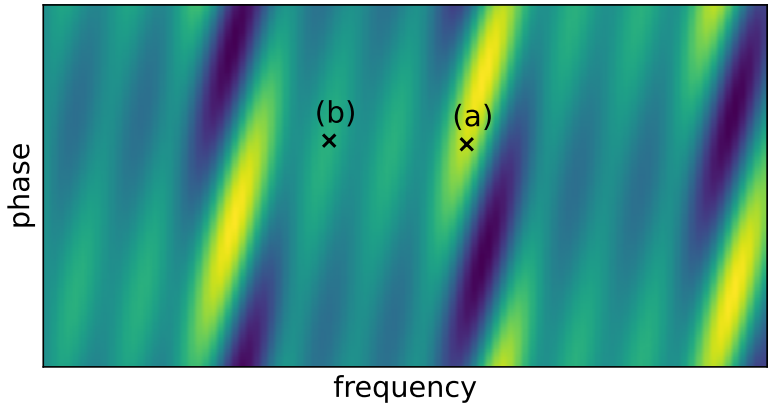
Finally, we trained three models with different seeds on the full validation data and tested on GTZAN [35], the held-out test set. From this, we computed a set of test metrics with statistical average and error.



(a) A wave with soft recall 0.92, close to a global maximum.



(b) A wave with soft recall 0.62, stuck at a local maximum.



(c) Soft Recall for a window with four ground truth beats, as a function of frequency and phase. There are bright maxima at integer multiples of the true frequency, where 100% soft recall can be achieved. There are also local maxima for bad grids, for example at (b).

Figure 3.2: Showcase of the properties of soft recall, applied to a single window. True beat positions are shown with red dashed lines in (a) and (b). Soft recall is differentiable with respect to phase and frequency of the predicted wave, but has local maxima with imperfect score as shown in (c), which means that soft-recall is non-concave.

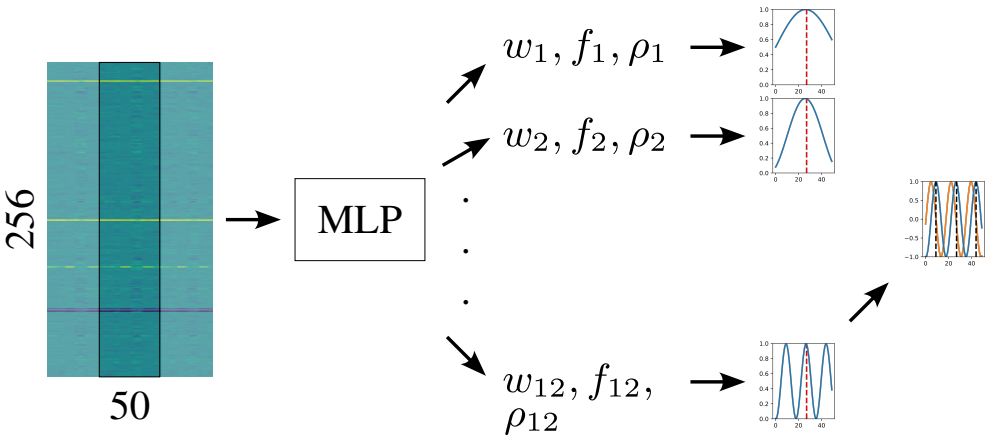


Figure 3.3: For a 1-second window of features (a), the grid block estimates weight, frequency and phase of 12 tatum candidates (b). Each defines a sine wave trained to maximize soft recall (c). The highest weighted one defines the tatum points and is passed on as a sine/cosine feature (d).

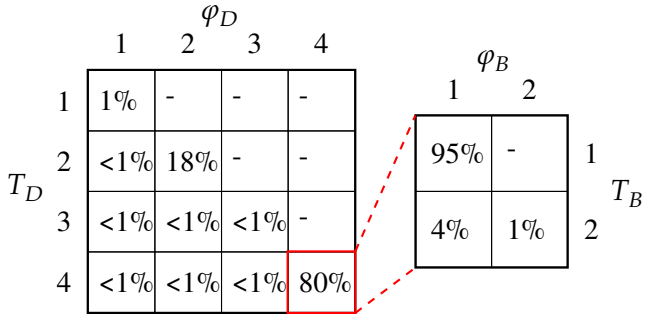


Figure 3.4: Illustration of a distribution over the subgrid state-space, with reduced dimensionality of T_B and T_D.

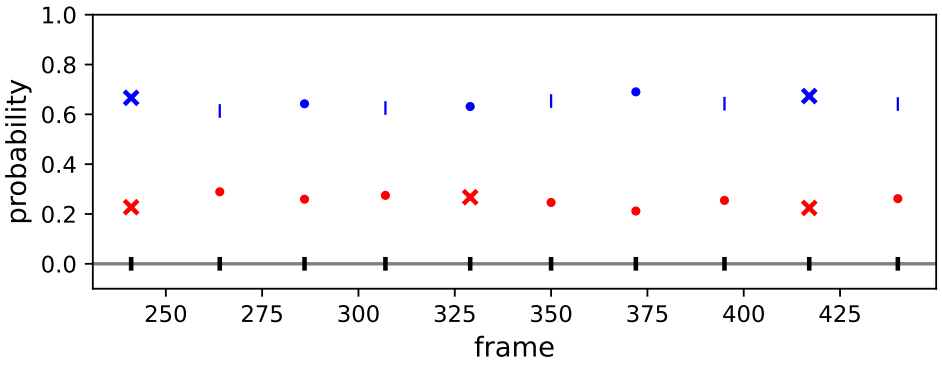


Figure 3.5: Visualization of the most probable subgrid states for the two most probable metric signatures. Shown in blue is [two tatums per beat, four beats per downbeat], and in red is [one tatum per beat, four beats per downbeat]. Only the most probable phase is shown in each case. Downbeats are marked with $\times$, beats with $\cdot$ and offbeats with $|$. The markings at $y = 0$ denote the tatum grid.

Chapter 4

Results and Analysis

In this section, we will discuss the performance of the tatum grid and the final output of the model. The tatum grid is compared to a non-learned way of estimating a tatum grid on the cross-validation set: PLP [16]. For the final beat and downbeat estimates, we compared our model to the models listed in Section 2.4.1.

4.1 Major results

We first present the results for the tatum grid on the cross validation and final data, then we present the beat and downbeat results, which we compare to the existing state-of-the-art.

4.1.1 Grid evaluation

In Table 4.1, we report the metrics for the grid in the cross-validation experiment. We can see that recall hovers around 90% for the majority of the datasets, with a notably worse performance on ASAP and SMC that contain classical music, and JAAH that contains jazz. Interestingly, AMLt is slightly higher than recall on JAAH and HJDB, which suggests that the estimated tatum grid is too slow for some pieces in these datasets. The recall is very good on the pop-focused datasets like Beatles and Harmonix, which is to be expected as these often feature a high-hat acting as an audible tatum-grid. Classical music often features varying timings and tempos as used as a method of expression, which can explain the difficulties in describing it with a consistent rhythmic grid.

For the PLP comparison, we used an energy based onset function and

computed the PLP with a window size of 6 seconds, a hop length of 2 seconds, and a bpm range of 40-480. These values were chosen based on the performance on the full training and validation set. Still, PLP performs far worse than our grid, especially in datasets where many excerpts lack drums. This should not be taken as the best possible performance achievable with PLP, as such a task would be worthy of a paper in itself, and it is likely that performance would be different with dPLP [17]. However, it serves as a starting point for future comparisons. It would be very useful to compare to the PLP in the pretraining stage of [18], unfortunately that is not possible as the PLP parameters or onset function they use are not published.

Table 4.1: Grid metrics on cross validation dataset. Grid re- call is measured as for the F1-score, by counting how many annotated beats have a grid point within 70 ms. AMLt is calculated treating the grid points as beat predictions. Scores are reported from our models predicted grid as well as a non-learned PLP algorithm.

	Recall		AMLt	
	Ours	PLP	Ours	PLP
ASAP	80.9	60.0	56.3	8.22
Ballroom	96.8	56.6	89.9	24.0
Beatles	97.3	56.5	89.0	23.9
Candombe	99.3	85.6	97.7	5.76
Filosax	93.0	56.8	91.5	25.0
Gr. MIDI	92.4	71.6	87.4	9.61
GuitarSet	89.3	68.4	83.4	18.8
Hainsworth	93.3	59.4	90.9	18.2
Harmonix	96.9	67.5	92.6	20.9
HJDB	95.8	71.1	96.6	30.3
JAAH	84.1	54.3	85.6	25.5
RWC	90.1	60.0	80.2	14.6
Simac	87.6	61.4	82.6	9.33
SMC	69.1	45.1	51.0	8.75
TapCorrect	96.6	62.4	85.9	12.8
All	91.1	62.1	83.5	17.1

On the test set, the grid achieved a recall of around 90%, as seen in Table 4.2. The AMLt is not notably higher or lower than the recall, which gives us

little to no additional information. This is somewhat expected as the test set contains a range of genres that tend to be difficult for the grid in different ways. The PLP score was not computed for the test set, as it performed poorly on the validation data. Presenting the PLP score alongside the tatum grid would suggest that the our strategy performs fantastically, when it should rather only be used as a baseline for future reference.

Table 4.2: Results for the tatum grid on GTZAN.

Recall	AMLt
90.5 ± 2.0	89.6 ± 0.6

4.1.2 Beat and Downbeat Tracking evaluation

For the cross validation, we compare our model to the Beat This! model by Foscarin *et al.* [2] as the open source implementation allows us to run direct experiments. For our model, we provide results for both the Threshold and Viterbi methods of inference. The results can be found in Table 4.3. Here, we see that our model is generally outperformed when it comes to beats, but has a significantly better score than Foscarin *et al.,* on downbeats for almost all datasets. Interestingly, Threshold inference generally outperforms Viterbi.

On the cross-validation data it is clear that the performance differs wildly depending on genre. Another insightful split of the data is to separate by time signatures. We split the data into *pure* time signatures, that are unchanged throughout the entire piece, *changing* time signatures that change somewhat often, and *leftovers* that have occasional time signature changes. The term time signature here is slightly misleading, as the ground truth annotations do not directly contain information about time signature, only about bar lengths. We use the term time signature as it is more familiar to the average reader, and a time signature change almost always corresponds to a bar length change. The results on the splits are shown in Table 4.4. We can compare directly to Beat This as we have access to per-song metrics for the entire cross-validation run. The results on this split highlights the class imbalance when it comes to time signatures and how much worse the models perform on non 4/4 music.

The results on the test set, shown in Table 4.5, show beat metrics that are slightly below the state-of-the-art. For downbeats however, our model gives significantly better continuity scores than other DBN-free methods. Including DBN-models in the comparison, the DBN-variant of [18] by Gagneré *et al.,*

Table 4.3: Results of 8-fold cross validation. We compare our model using threshold inference (Thr.), our model using Viterbi inference (Vit.) and the Beat This model (BT).

	Beat CMLt			Downbeat CMLt		
	Thr.	Vit.	*BT*	Thr.	Vit.	*BT*
ASAP	**50.4**	50.1	50.3	**31.2**	28.6	23.5
Ballroom	92.8	90.6	**96.4**	**95.0**	92.5	92.9
Beatles	84.8	83.3	**87.2**	**77.8**	76.1	73.7
Candombe	98.9	98.3	**99.8**	99.6	99.3	**99.9**
Filosax	85.5	84.9	**98.8**	**98.7**	97.7	96.1
Gr. MIDI	82.9	83.4	**84.9**	**77.5**	76.8	70.9
GuitarSet	**85.0**	84.5	82.4	**89.6**	86.9	79.8
Hainsworth	81.2	81.4	**84.0**	73.4	**73.6**	63.6
Harmonix	88.5	87.9	**89.9**	**85.2**	84.5	81.2
HJDB	94.6	94.6	**97.2**	94.2	94.3	**94.8**
JAAH	68.5	68.1	**88.5**	**78.8**	78.3	71.5
RWC	74.3	73.3	**75.7**	**72.3**	70.6	69.0
Simac	**72.7**	72.6	55.8	-	-	-
SMC	48.0	48.5	**51.4**	-	-	-
TapCorrect	**82.2**	79.8	81.9	**79.3**	74.8	68.2
All	**79.2**	78.5	79.0	**78.4**	76.9	73.5

performs slightly better than our model on all metrics. On the test set, Viterbi inference performs better than Threshold.

4.1.3 Qualitative analysis

The grid generally handles common music genres and styles well. It generally struggles with music that breaks the assumption of a steady tatum, like the previously discussed rubato in classical music, and excerpts with very fast tempos. Some of these issues are shown in Figure 4.1. The grid almost always represents the beat or twice the tempo of the beat, though there is nothing that specifically enforces this in the architecture.

The subgrid learned to predict phase for the common time signatures like 4/4 and 3/4, but almost never predicted abrupt time signature changes. This is apparent on many songs in the Beatles dataset that feature measures of 2/4 in an otherwise 4/4 song, shifting the downbeat. On such pieces (like the one

Table 4.4: Results on different subsets of cross validation data, excluding datasets without downbeat annotations. Pure refers to samples with constant number of beats per downbeat. 4/4 refers to samples with 4 beats per downbeat. Changing refers to samples with more than two bar length changes per minute. Leftovers are non-pure pieces with fewer than two bar length changes per minute.

	Pure 4/4	Pure non-4/4	Changing	Leftovers
Share of data	75.7%	13.3%	1.6%	9.3%
No. of pieces	2836	499	59	350
	Downbeat CMLt			
Threshold	**84.0**	**56.0**	46.9	**69.9**
Viterbi	82.6	54.3	**47.3**	67.8
Beat This [2]	78.9	54.4	36.4	63.6

in Figure 4.2, the subgrid generally just slowly shifted its confidence from one phase offset to another, instead of actually predicting a bar of 2/4. The Viterbi decoding means that either a 2/4 or a 6/4 measure will be generated when confidence in the new phase is high enough, but it is not a direct prediction.

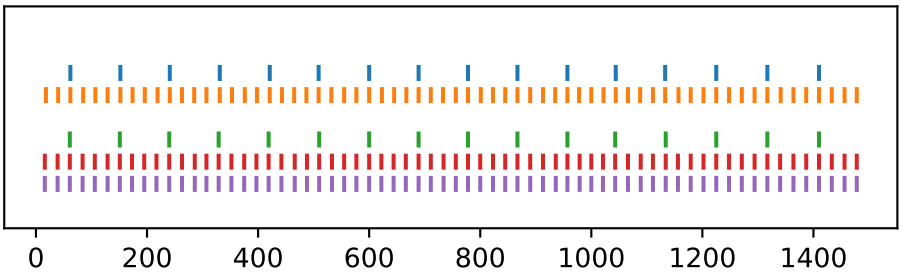
The subgrid also favored estimating probabilities very close to 0, even for many true beats in very reasonable places.

4.2 Reliability Analysis

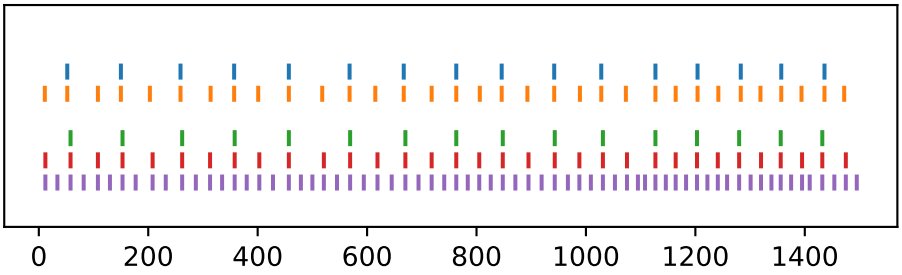
For the training of our model, we have used identical data and training setup as Foscarin *et al.*, which makes direct comparison to their model very reliable. We have also used standard metrics and testing datasets when applicable, so the comparison to other beat tracking models are direct. A bias exists in the choice of Beat This! as the base model platform, as that choice was impacted by the models good scores on the test dataset. However, we believe that our architecture is different enough to not carry over much of an implicit overfitting to the test data. We also provided error margins on the metrics, that we calculated by training multiple models with different seeds. All hyperparameters were determined in the validation phase, before the test data was visible to the model.

Table 4.5: Results on GTZAN for the final model, compared with state-of-the-art models with and without DBN postprocessing. The best score in each metric is marked in **bold** for each of the two categories.

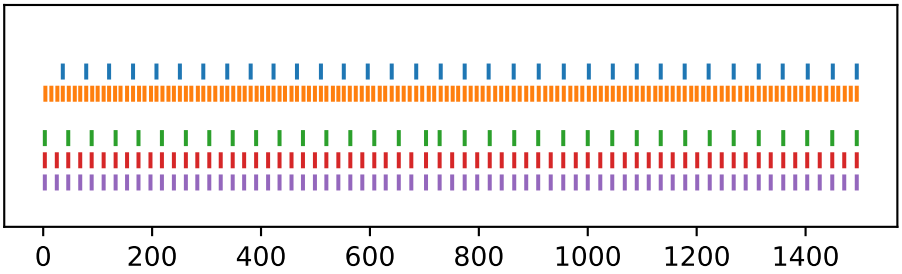
Beat	F1	CMLt	AMLt
DBN-free			
Our (Threshold)	87.9 ± 0.8	79.8 ± 1.2	90.3 ± 0.1
Our (Viterbi)	88.0 ± 0.7	80.0 ± 1.1	90.4 ± 0.1
Beat This [2]	89.1 ± 0.3	79.8 ± 0.6	89.8 ± 0.4
Gagneré et al. [18] ST-BCE	**89.6**	**81.8**	**91.8**
With DBN			
Beat This + DBN	88.1 ± 0.3	80.5 ± 0.4	91.1 ± 0.2
Hung et al. [10]	88.7	81.2	92.0
Gagneré et al. BCE-DBN	**89.6**	**82.6**	**92.5**
Downbeat	F1	CMLt	AMLt
DBN-free			
Our (Threshold)	77.3 ± 0.9	72.4 ± 0.7	85.1 ± 0.6
Our (Viterbi)	77.4 ± 1.0	**72.7** ± 0.8	**86.2** ± 0.4
Beat This	**78.3** ± 0.4	67.3 ± 0.8	79.1 ± 0.6
Gagneré et al. ST-BCE	77.5	67.0	80.8
With DBN			
Beat This + DBN	77.4 ± 0.2	73.3 ± 0.2	87.8 ± 0.5
Hung et al.	75.6	71.5	88.1
Gagneré et al. BCE-DBN	**78.3**	**74.7**	**88.2**



(a) Model output on an excerpt from Beatles [1]. The tatum grid is the same as the beat, and the prediction is good.

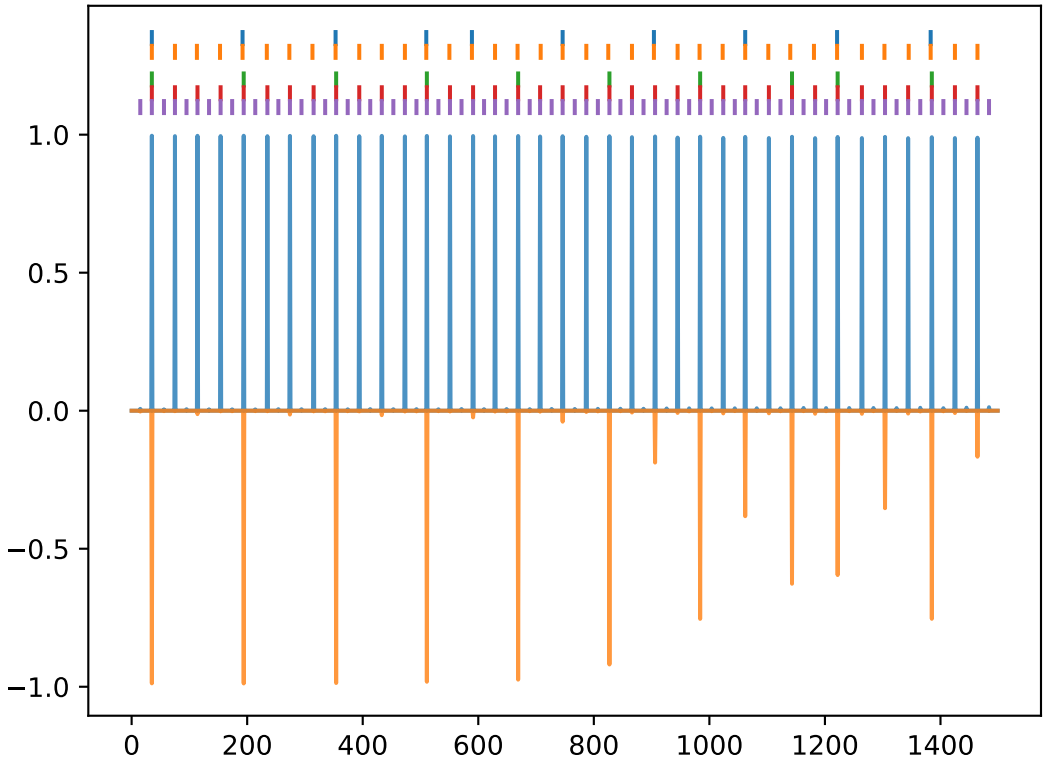


(b) Output on an excerpt from ASAP [20]. The tatum grid is faster than the beat. Around frame 1100, the tempo slows down and the grid speeds up to align itself again when the tempo returns to normal. This results in an extra tatum pulse, which is bad as the subgrid has to break the rules to predict the correct beats there.

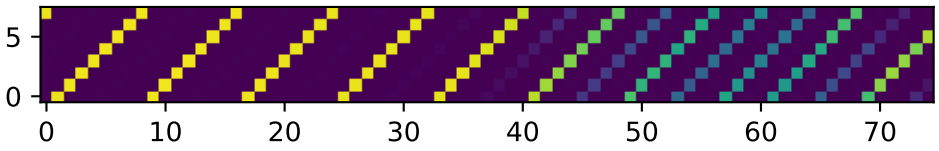


(c) Output on a fast jazz excerpt from JAAH [30]. The excerpt has an average tempo of 272 bpm, and the grid is too slow.

Figure 4.1: A showcase of different grid estimations (with estimated beats and downbeats included). The pulse trains represents, from top to bottom: true downbeats, true beats, estimated downbeats, estimated beats and estimated tatum grid points.



(a) Full prediction on an excerpt from Cry Baby Cry by the Beatles. The blue lines between 0 and 1 represent local beat probability, while the orange lines between 0 and -1 represent downbeat probabilities. The song features a measure of 2/4 that leads to the downbeat being offset. The model is slow to adapt to this, and estimates a very low probability of that being the true downbeat in the beginning.



(b) Subgrid phase distribution for the metric signature $(T_B, T_D) = (2, 4)$. The x-axis represents grid point, and y axis represents the different (φ_B, φ_D) pairs, ordered such that the y=0 represents the downbeat. For this excerpt, the estimated probability of all other metric signatures were near zero.

Figure 4.2: Illustrative example of how the subgrid fails to switch time signature, instead just shifting to a new phase.

Chapter 5

Discussion

The model performs quite well, with a significant jump in downbeat CMLt compared to the previous state-of-the-art for DBN-free models. Surprisingly, the Threshold inference seems to perform consistently better on the cross validation while the Viterbi inference does better on the test set. This can be explained in multiple ways.

Firstly, the inability of Viterbi inference to perform better could be a sign that having a subgrid distribution does not provide value over having a simple beat activation. A more positive outlook would be to say that the subgrid regularization works so well that it is trained to implicitly mimic the Viterbi results, which is already baked into the activations. This is supported by the strong performance on downbeat CMLt. Thirdly, by looking qualitatively at the subgrid outputs, it is clear that it is not really doing what it was designed to do. It heavily favors 4/4 time signatures to the point where it sometimes predicts 3/4 by just predicting a 4/4 with jumps in every measure. This overfitting to the common time signature is partly a symptom of the extreme class imbalance in the dataset, but is also a sign that the regularization does not punish jumps enough in the subgrid. This is difficult to combat during training, because simply weighing the regularization loss higher can also lead to the subgrid learning to only predict 4/4 consistently and just ignore the few pieces where that results in errors.

The subgrid had larger problems, largely caused by training it with MSE instead of BCE. BCE is designed to work with probabilities while MSE is not, and BCE would likely work better for splitting confidence into multiple metric signatures, which could lead to better training and better performance, especially when using Viterbi. Using MSE was not our intention, but rather a workaround to nan-errors during training when using BCE that we did not

have time to investigate. This was likely due to poor programming, and we believe that switching to BCE could yield great results for the subgrid actually doing its intended task.

As for the grid, it mostly performs well. A recall of 90% is lower than expected however, and improvements to the grid could certainly contribute to a much stronger model. The inability of the grid to quickly change tempo is caused by the windowed approach and the window size. A smaller window size resulted in issues with training, so a solution could be to allow the model to predict some extra parameters for each candidate tatum wave to control pauses and timing.

Chapter 6

Conclusions and Future work

We will discuss what the model achieved, and how it relates to the goals of the project. Then we will give suggestions about which parts have highest potential for impact and should be explored.

6.1 Conclusions

The first goal was to teach the model to predict a tatum grid, and it performed far better than our simple PLP implementation. Still, we did not manage to design a grid module that could fully capture rubato or other timing variations, and it was somewhat prone to overfitting to slower tempos, sometimes estimating tatum grids slower than the actual bpm of the piece. These are most likely problems that could be remedied with a better design of the grid module. For example, generating the tatum grid in shorter windows could make it easier for the model to handle timing changes, if it can be done without training collapsing. Another idea is to change the shape of the sinusoid used in the soft recall to a wave with wider peaks. This could decrease the incentive to keep the tatum grid slow, and perhaps improve performance on fast pieces.

The second goal was to create a proof-of-concept model using our proposed architecture and have it perform at a level similar to current state-of-the-art models, and this was also successful. While worse than the best DBN-models, the good downbeat CMLt scores indicate that it is better at internal consistency and correctness than other non-DBN models, which is important as the DBN enforces many constraints that do not apply to all music.

We proved that the initial grid-subgrid architecture works and can perform on par with beat-activation based architectures with many years of iteration and research. This is a good sign that similar polish could result in a much

better performance for grid-subgrid architectures.

6.2 Future work

We believe that it should be possible to work independently on improving the grid module and the subgrid module. Papers focusing on the grid can directly target grid metrics, while subgrid modules should be able to be swapped out and tested with the same grid for the sake of comparison.

A thorough exploration of the tatum grid could be helped by designing metrics that better capture whether the grid keeps a constant subdivision of the beat. Alternative ways to produce grid hypotheses could be explored, perhaps using self-contrastive learning instead of the separate tempo-ranges of our model. To perform better on classical music, the timing variation problem would need to be solved, and it seems unlikely that shorter windows will suffice.

The subgrid did not always do what we expected, and this could certainly be explored further. Firstly, getting it working with BCE should make it much better at predicting probabilities correctly. Secondly, the subgrid does not seem to learn how to change time signatures at a downbeat, instead just learning to slowly shift the estimated distribution to a different phase when it wants to move the downbeat. This should be explored further.

Model design aside, many issues stem from the class imbalance in the training data. This is true both in terms of predicting outlier tempos (Figure 4.1) and rare time signatures, which are rare in the training data precisely because they are rare in music. It would be interesting to focus on the performance on such pieces, and see if it can be improved through more advanced data augmentation or weighing the loss on uncommon types of music higher during training. Another approach would be to simply annotate more pieces with challenging time signatures, or generating audio based on MIDI tracks designed with challenging features.

6.3 Reflections

This project started with an idea: Beat Tracking should be best described as a distribution over metric interpretations. The rest of the model architecture and design came from that simple seed, and many of the design choices felt more like logical consequences than choices between equal options. This makes it particularly interesting to see where other people can find alternative solutions

or designs.

Whether this leads to a new path forward for Beat Tracking or ends up as an interesting dead end, we believe that the contributions and insights of this project are useful for the field, and hope that others continue pushing the boundaries forward.

References

[1] M. E. P. Davies, N. Degara, and M. D. Plumbley, "Evaluation methods for musical audio beat tracking algorithms," *Queen Mary University of London, Centre for Digital Music, Tech. Rep. C4DM-TR-09-06*, 2009.

[2] F. Foscarin, J. Schlüter, and G. Widmer, "Beat this! accurate beat tracking without DBN postprocessing," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2024.

[3] R. Kihlborg, A. Holzapfel, and J. Schlüter, "Explore this! beat and downbeat tracking from a learned tatum grid," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2026.

[4] J. Bilmes, "Timing is of the essence: Perceptual and computational techniques for representing, learning, and reproducing expressive timing in percussive rhythm," Master's thesis, MIT, Cambridge, MA, 1993.

[5] R. Polak, "Rhythmic feel as meter: Non-isochronous beat subdivision in jembe music from mali," *Music Theory Online*, vol. 16, no. 4, 2010.

[6] M. Goto and Y. Muraoka, "A beat tracking system for acoustic signals of music," in *Proceedings of the Second ACM International Conference on Multimedia '94*. San Francisco, CA, USA: ACM Press, 1994, pp. 365–372.

[7] F. Krebs, S. Böck, and G. Widmer, "An efficient state-space model for joint tempo and meter tracking," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2015.

[8] S. Böck, F. Krebs, and G. Widmer, "Joint beat and downbeat tracking with recurrent neural networks," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2016.

[9] S. Böck and M. E. P. Davies, "Deconstruct, analyse, reconstruct: How to improve tempo, beat, and downbeat estimation," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2020.

[10] Y.-N. Hung, J.-C. Wang, X. Song, W.-T. Lu, and M. Won, "Modeling beats and downbeats with a time-frequency transformer," in *Proceedings of the International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2022.

[11] J. Zhao, G. Xia, and Y. Wang, "Beat transformer: Demixed beat and downbeat tracking with dilated self-attention," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2022.

[12] T. Chen and L. Su, "Toward postprocessing-free neural networks for joint beat and downbeat estimation," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2022.

[13] J. Seppänen, "Computational problems of musical meter recognition," Master's thesis, Tampere University of Technology, 2001.

[14] C. Uhle and J. Herre, "Estimation of tempo, micro time and time signature from percussive music," in *Digital Audio Effects (Dafx)*, 2003.

[15] S. Durand, J. P. Bello, B. David, and G. Richard, "Downbeat tracking with multiple features and deep neural networks," in *Proceedings of the International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2015. doi: 10.1109/ICASSP.2015.7178001

[16] P. Grosche and M. Müller, "Extracting predominant local pulse information from music recordings," *IEEE Trans. Speech Audio Process.*, vol. 19, no. 6, pp. 1688–1701, 2011.

[17] C. Chiu, S. Strahl, and M. Müller, "dplp: A differentiable version of predominant local pulse estimation," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2025.

[18] A. Gagneré, S. Essid, and G. Peeters, "Controlling contrastive self-supervised learning with knowledge-driven multiple hypothesis: Application to beat tracking," *CoRR*, vol. abs/2510.25560, 2025. doi: 10.48550/ARXIV.2510.25560. [Online]. Available: https://doi.org/10.48550/arXiv.2510.25560

[19] M. Cuturi, "Sinkhorn distances: Lightspeed computation of optimal transportation distances," 2013. [Online]. Available: https://arxiv.org/abs/1306.0895

[20] F. Foscarin, A. Mcleod, P. Rigaux, F. Jacquemard, and M. Sakai, "ASAP: a dataset of aligned scores and performances for piano transcription," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2020.

[21] F. Gouyon, "A computational approach to rhythm description — Audio features for the computation of rhythm periodicity functions and their use in tempo induction and music content processing," Ph.D. dissertation, Universitat Pompeu Fabra, 2006.

[22] F. Krebs, S. Böck, and G. Widmer, "Rhythmic pattern modeling for beat and downbeat tracking in musical audio," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2013.

[23] M. Rocamora, L. Jure, B. Marenco, M. Fuentes, F. Lanzaro, and A. Gómez, "An audio-visual database of Candombe performances for computational musicological studies," in *Congreso Internacional de Ciencia y Tecnología Musical (CICTeM)*, 2015.

[24] D. Foster and S. Dixon, "Filosax: A dataset of annotated jazz saxophone recordings," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2021.

[25] J. Gillick, A. Roberts, J. Engel, D. Eck, and D. Bamman, "Learning to groove with inverse sequence transformations," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2019.

[26] Q. Xi, R. M. Bittner, J. Pauwels, X. Ye, and J. P. Bello, "GuitarSet: A dataset for guitar transcription." in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2018.

[27] S. W. Hainsworth and M. D. Macleod, "Particle filtering applied to musical tempo tracking," *EURASIP Journal on Advances in Signal Processing*, vol. 2004, pp. 1–11, 2004.

[28] O. Nieto, M. C. McCallum, M. E. P. Davies, A. Robertson, A. M. Stark, and E. Egozy, "The Harmonix set: Beats, downbeats, and functional

segment annotations of western popular music." in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2019.

[29] J. Hockman, M. E. P. Davies, and I. Fujinaga, "One in the jungle: Downbeat detection in hardcore, jungle, and drum and bass." in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2012.

[30] V. Eremenko, E. Demirel, B. Bozkurt, and X. Serra, "Audio-aligned jazz harmony dataset for automatic chord transcription and corpus-based research," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2018.

[31] M. Goto, H. Hashiguchi, T. Nishimura, and R. Oka, "RWC music database: Popular, classical and jazz music databases," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2002.

[32] M. Goto, "AIST annotation for the RWC music database," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2006.

[33] A. Holzapfel, M. E. P. Davies, J. R. Zapata, J. L. Oliveira, and F. Gouyon, "Selective sampling for beat tracking evaluation," *Transactions on Audio, Speech, and Language Processing*, vol. 20, no. 9, pp. 2539–2548, 2012.

[34] J. Driedger, H. Schreiber, W. B. de Haas, and M. Müller, "Towards automatically correcting tapped beat annotations for music recordings." in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2019.

[35] U. Marchand and G. Peeters, "Swing ratio estimation," in *Digital Audio Effects (Dafx)*, 2015.

[36] C. Raffel, B. McFee, E. J. Humphrey, J. Salamon, O. Nieto, D. Liang, D. P. Ellis, and C. C. Raffel, "mir_eval: A transparent implementation of common MIR metrics," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2014.

TRITA – XXX-EX 2025:0000
Stockholm, Sweden 2026

www.kth.se

€€€€ For DIVA €€€€

{
”Author1”: { ”Last name”: ”Kihlborg”,
”First name”: ”Robert”,
”Local User Id”: ”u1okbj0u”,
”E-mail”: ”robertok@kth.se”,
”organisation”: {”L1”: ”School of Electrical Engineering and Computer Science”,
}
},
”Cycle”: ”2”,
”Course code”: ”DA233X”,
”Credits”: ”30.0”,
”Degree1”: {”Educational program”: ”Master’s Programme, Machine Learning, 120 credits”
,”programcode”: ”TMAIM”
,”Degree”: ”Degree of Master of Science in Engineering”
,”subjectArea”: ”Machine Learning”
},
”Title”: {
”Main title”: ”Latent grid distributions for better robustness in Beat Tracking”,
”Subtitle”: ”Encoding ambiguity with metric distributions over a learned tatum grid”,
”Language”: ”eng” },
”Alternative title”: {
”Main title”: ”Underliggande fördelningar över rytmiska tolkningar för robustare taktanalys”,
”Subtitle”: ”Beskrivningar av tvetydlighet genom fördelningar över rytmik ovanpå en inlärd puls”,
”Language”: ”swe”
},
”Supervisor1”: { ”Last name”: ”Holzapfel”,
”First name”: ”Andre”,
”Local User Id”: ”u1XXXXXX”,
”E-mail”: ”holzap@kth.se”,
”organisation”: {”L1”: ”School of Electrical Engineering and Computer Science”,
”L2”: ”Media Technology & Interaction Design” }
},
”Supervisor2”: { ”Last name”: ”Schlüter”,
”First name”: ”Jan”,
”E-mail”: ”schlueter@jku.at”,
”Other organisation”: ”Johannes Kepler University, Department of Computational Perception”
},
”Examiner1”: { ”Last name”: ”Sturm”,
”First name”: ”Bob”,
”Local User Id”: ”u1XXXXXX”,
”E-mail”: ”bobs@kth.se”,
”organisation”: {”L1”: ”School of Electrical Engineering and Computer Science”,
”L2”: ”Speech, Music & Hearing” }
},
”National Subject Categories”: ”10210, 10106, 60412”,
”Other information”: {”Year”: ”2026”, ”Number of pages”: ”ix,43”},
”Copyrightleft”: ”copyright”,
”Series”: { ”Title of series”: ”TRITA – XXX-EX” , ”No. in series”: ”2025:0000” },
”Opponents”: { ”Name”: ”XXX”},
”Presentation”: { ”Date”: ”2022-03-15 13:00”
,”Language”:”XXX”
,”Room”: ”via Zoom https://kth-se.zoom.us/j/ddddddddddd”
,”Address”: ”Isafjordsgatan 22 (Kistagången 16)”
,”City”: ”Stockholm” },
”Number of lang instances”: ”2”,
”Abstract[eng]”: €€€€


```
% Enter abstract here, 250-350 words roughly.

% key parts of the abstract
% \begin{itemize}
%   \item What is the topic area? (optional) Introduces the subject area for the project.
%   \item Short problem statement
%   \item Why was this problem worth a Bachelor's/'Masters thesis project? (\ie, why is the problem
both significant and of a suitable degree of difficulty for a Bachelor's/'Masters thesis project? Why
has no one else solved it yet?)
%   \item How did you solve the problem? What was your method/insight?
%   \item Results/Conclusions/Consequences/Impact: What are your key
results/\linebreak[4]conclusions? What will others do based on your results? What can be done now
that you have finished - that could not be done before your thesis project was completed?
% \end{itemize}

Beat Tracking is the task of predicting the positions of beats and/or downbeats in a musical
recording. The current state of the art uses Deep Learning models on spectrogram representations of
music to estimate beat and downbeat positions. Usually, the networks produce a beat activation curve
that represents the probability of a beat, and then post-process that curve to get the final
estimates. To ensure musical consistency, this is often done with Dynamic Bayesian Network (DBN)
```

postprocessing, which only works for certain types of music. Finding DBN-free ways to enforce musical consistency is desirable as it could expand the usefulness of Beat Tracking algorithms. This thesis explores novel ways of using theory-informed modules in tandem with Deep Learning to facilitate learning of metrical consistency. The proposed model predicts a tatum grid: a regular grid of feasible beat candidates, and a subgrid: a regular subset of the tatum grid that constitutes the final estimation. The subgrid treats meter like a classification problem, allowing for multiple hypotheses for the time signature and tempo, which can better represent ambiguity than a prediction based on local beat probabilities. The trained model improves continuity metrics compared to state-of-the-art DBN-free algorithms, though it still performs worse than DBN-methods on the test dataset. Despite good scores, qualitative analysis shows the model sometimes failing on easy data, suggesting that the algorithm has a lot of room for improvement.

"Abstract[swe]": €€€€

% *Beat Tracking is the task of predicting the positions of beats and/or downbeats in a musical recording. The current state-of-the-art in the field uses Deep Learning models on spectrogram representations of music to estimate beat and downbeat positions. Usually, the networks produce a beat activation curve that represents the probability of a beat, and then post-process that curve to get the final estimates. To ensure musical consistency, this is often done with Dynamic Bayesian Network (DBN) postprocessing, which uses assumptions that are false for entire genres of music. Finding DBN-free ways to enforce musical consistency is desirable as it could expand the usefulness of Beat Tracking algorithms.*
% *This thesis explores novel ways of using theory-informed modules in tandem with Deep Learning to facilitate learning of metrical consistency. The proposed model predicts a tatum grid: a regular grid of feasible beat candidates, and a subgrid: a regular subset of the tatum grid that constitutes the final estimation. The subgrid treats meter like a classification problem, allowing for multiple hypotheses for the time signature and tempo, which can better represent ambiguity than a prediction based on local beat probabilities. The trained model improves continuity metrics compared to state-of-the-art DBN-free algorithms, though it still performs worse than DBN-methods on the test dataset. Despite good scores, qualitative analysis shows the model sometimes failing in strange ways on easy data, suggesting the existence of low hanging fruits that could improve the algorithm further.*

Taktanalys går ut på att automatiskt hitta takten i inspelad musik. De bästa algoritmerna idag transformerar musiken till spektrogram och använder dessa som indata till djupinlärningsmodeller som försöker lära sig att placera ut taktslag på rätt ställen. Vanligtvis görs detta genom att först producera en signal som vid varje tidpunkt representerar den lokala sannolikheten för ett taktslag, som sedan kan omvandlas till en fullständig gissning. För att omvandlingen mellan taktsignalen och slutgiltiga gissningen ska vara musikaliskt konsekvent används ofta Dynamiska Bayesianska Nätverk (DBN) som kan garantera konsekventa uppskattningar givet begränsningar inom exempelvis tillåtna tempon eller taktarter. Genom att göra sådana antaganden så kan modellens snittprestanda höjas, men det innebär också att hela genrer av musik exkluderas. Därför är det värdefullt att hitta DBN-fria algoritmer som kan förutspå konsekventa takter.
Det här examensarbetet utforskar nya sätt att använda moduler inspirerade av musikteori tillsammans med djupinlärning för att underlätta inlärning av konsekventa taktgissningar. Den framtagna modellen börjar med att förutspå ett tatumnät: en regelbunden serie av taktkandidater, och ett delnät: en regelbunden delmängd av tatumnätet som utgör den slutgiltiga gisssningen. Delnätet tolkar taktart som ett klassifikationsproblem vilket möjliggör för flera hypoteser för tempo och taktart, vilket gör den bättre på att uppskatta tvetydligheter än DBN-modeller som bara producerar en tolkning. Modellen presterar bättre än andra DBN-fria modeller på tester som mäter konsekvent korrekthet, men presterar sämre än de bästa DBN-modellerna i snitt. Trots bra mätvärden visar den kvalitativa analysen att modellen har oväntade svårigheter på relativt enkelt indata, vilket antyder att algoritmen har enkla förbättringsmöjligheter.

€€€€,
"Keywords[eng]": €€€€
Beat Tracking, Music Informatics, Machine Learning €€€€,
€€€€,
"Keywords[swe]": €€€€
Taktanalys, Musikinformatik, Maskininlärning €€€€,
}



acronyms.tex

```
%%% Local Variables:
%%% mode: latex
%%% TeX-master: t
%%% End:
% The following command is used with glossaries-extra
\setabbreviationstyle[acronym]{long-short}
% The form of the entries in this file is \newacronym{label}{acronym}{phrase}
%                                      or \newacronym[options]{label}{acronym}{phrase}
% see "User Manual for glossaries.sty" for the  details about the options, one example is shown below

\newacronym{DBN}{DBN}{Dynamic Bayesian Network}
\newacronym{DAW}{DAW}{Digital Audio Workstation}
\newacronym{STFT}{STFT}{Short Time Fourier Transform}
\newacronym{HMM}{HMM}{Hidden Markov Model}
\newacronym{NN}{NN}{Neural Network}
\newacronym{NLP}{NLP}{Natural Language Processing}
\newacronym{KTH}{KTH}{KTH Royal Institute of Technology}
\newacronym{JKU}{JKU}{Johannes Kepler University Linz}
\newacronym{MIR}{MIR}{Music Information Retrieval}
\newacronym{CNN}{CNN}{Convolutional Neural Network}
\newacronym[longplural={Long-Short Term Memory networks}]{LSTM}{LSTM}{Long-Short Term Memory}
\newacronym{DL}{DL}{Deep Learning}
\newacronym[longplural={Mixture-of-Experts models}]{MoE}{MoE}{Mixture-of-Experts}
\newacronym{TCN}{TCN}{Temporal Convolutional Network}
\newacronym{PLP}{PLP}{Predominant Local Pulse}
\newacronym{dPLP}{dPLP}{differentiable Predominant Local Pulse}
\newacronym{EMD}{EMD}{Earth Mover's Distance}
\newacronym{MSE}{MSE}{Mean Squared Error}
\newacronym{BCE}{BCE}{Binary Cross-Entropy}

% EXAMPLES BELOW:

% note the specification of the long form plural in the line below
% \newacronym[longplural={Debugging Information Entities}]{DIE}{DIE}{Debugging Information Entity}
%
% The following example also uses options
% \newacronym[shortplural={OSes}, firstplural={operating systems (OSes)}]{OS}{OS}{operating system}

% note the use of a non-breaking dash in long text for the following acronym
% \newacronym{IQL}{IQL}{Independent -QLearning}

% example of putting in a trademark on first expansion

% \newacronym[first={NVIDIA OpenSHMEM Library (NVSHMEM\texttrademark)}]{NVSHMEM}{NVSHMEM}{NVIDIA OpenSHMEM Library}

% \newacronym{API}{API}{Application Programming Interface}
% \newacronym{DiVA}{DiVA}{Digitala Vetenskapliga Arkivet}
% \newacronym{IMRAD}{IMRAD}{Introduction, Methods, Results, and Discussione}
% \newacronym{JSON}{JSON}{JavaScript Object Notation}
% \newacronym{KI}{KI}{Karolinska Institutet}
% \newacronym{KOPPS}{KOPPS}{Kurs- och programplaneringssystemet}
% \newacronym{LADOK}{LADOK}{Lokalt -adbbaserat dokumentationssystemt}
% \newacronym{TIMTM}{TIMTM}{Interactive Media Technology}
% \newacronym{TMMTM}{TMMTM}{Media Managemen}
% \newacronym{KTH}{KTH}{KTH Royal Institute of Technology}

% \newacronym{LAN}{LAN}{Local Area Network}
% \newacronym{VM}{VM}{virtual machine}
% % note the use of a non-breaking dash in the following acronym
% \newacronym{WiFi}{-WiFi}{Wireless Fidelity}

% \newacronym{WLAN}{WLAN}{Wireless Local Area Network}
% \newacronym{UN}{UN}{United Nations}
% \newacronym{SDG}{SDG}{Sustainable Development Goal}
```

Empty list